Aufgabenblatt 1: Multimodale Routenplanung

Bearbeitungszeitraum: 19.10.2026 - 02.11.2026

Untersuchen Sie die Verwendbarkeit des A*-Algorithmus für eine multimodale Routenplanung in Berlin!

In einer csv-Datei finden Sie Informationen zum Berliner U-Bahn-Netz. Die Datei enthält alle Linien, Stationen, Fahrtdauern, Umsteigemöglichkeiten und Geokoordinaten der U-Bahn-Haltestellen. Für das Umsteigen werden pauschal 5 Minuten Zeit veranschlagt. Das Tempo zu Fuß beträgt 5 km/h.

Entwickeln Sie ein Programm in einer Programmiersprache Ihrer Wahl, das für beliebige Geokoordinaten in Berlin die zeitlich kürzeste Route berechnet und auf der Konsole ausgibt. Das Programm soll den A*-Algorithmus mit einer geeigneten Heuristik verwenden.

Eine Route besteht typischerweise aus 3 Reiseabschnitten: 1. Laufen zu einer nahen U-Bahn-Station[1], 2. U-Bahn fahren (ggfs. mit Umsteigen) zu einem U-Bahnhof nahe dem Zielort, 3. Laufen zur Zieladresse. Es gibt aber Situationen, in denen das Laufen vom Start zum Ziel, ohne Nutzung der U-Bahn, die optimale Lösung darstellt. Und nicht immer ist es optimal, den nächsten U-Bahnhof zum Zielort anzufahren, nämlich wenn durch Umsteigeaktionen zuviel Zeit verbraucht wird.

Das U-Bahn-Fahren besteht aus den einzelnen Stationen und den Umsteigeaktionen. Weil die längste U-Bahn-Linie knapp 40 Stationen enthält, kann die A*-Suche einen sehr tiefen Baum generieren. Um die Leistungsgrenzen der Suche bei Speicher- und Zeitlimitierungen auszutesten, implementieren Sie folgende Einschränkungen[2]:

(1) Begrenzung der maximalen Pfadlänge auf *p_max*[3]: Pfade mit größerer Länge werden nicht in die sortierte Liste eingefügt, die Suche geht aber weiter. Nach Terminierung des Algorithmus sind 3 Fälle zu unterscheiden: a) Die sortierte Liste ist leer, dann ist die Meldung "Lösung mit begrenztem Speicher nicht generierbar" auszugeben;
UPDATE: b) alle Pfade konnten expandiert werden (kein expandierter Pfad erreicht das *p_max*-Limit), dann wurde die optimale Lösung gefunden;
UPDATE: c) ~~der Lösungspfad hat die maximale Pfadlänge *p_max*~~, die Meldung "potenziell nicht optimale Lösung gefunden" wird ausgegeben, falls im Laufe der A*-Suche irgendein Zustand

[1] UPDATE: Das Laufen (als erste Aktion) sollte nur bis zu einer nahegelegenen U-Bahnstation erfolgen. Vermutlich ist es sinnvoll, alternativ oder zusätzlich eine maximale Reisedauer festzulegen, die beispielsweise das Laufen von Spandau nach Neukölln ausschließt.

[2] Geeignete Werte müssen Sie empirisch ermitteln. Sie sollen aber keine trivial kleinen Werte verwenden. Eine mögliche, aber vermutlich zu hohe Pfadlängenbegrenzung ist 40; ein sinnvolles Zeitlimit wird möglicherweise im niedrigen Sekundenbereich liegen, sollte aber 2 Minuten nicht überschreiten.

[3] Analog dem Verfahren der *beschränkten Tiefensuche*.

aufgrund von p_max nicht weiter expandiert wurde, die Suche aber mit einem Lösungspfad terminiert[4].

(2) Begrenzung der Suchdauer: Ihr Programm soll "hinreichend schnelle" Antwortzeiten garantieren. Setzen Sie daher ein sinnvolles Zeitlimit: Nach Erreichen dieses Zeitlimits terminiert Ihr Programm und liefert die beste gefundene Lösung, sofern vorhanden, wie in (1). Wurde keine Lösung gefunden, wird der Hinweis auf die Konsole geschrieben, dass die Suche mit Zeitlimit keinen Erfolg hatte.

Ihr Programm soll folgende Informationen auf die Konsole schreiben ("Logging"): Start- und Zielkoordinaten, Lösungspfad (sofern gefunden), Speicherverbrauch (maximale Zahl an Pfaden in der sortierten Liste), Anzahl untersuchter Zustände.

Experimente

Führen Sie folgende Experimente durch, um die Korrektheit des Programms, aber auch dessen Verhalten in Edge-Cases empirisch zu überprüfen.

Die Korrektheit ist anhand folgender Use-Cases zu zeigen (finden Sie jeweils Start- und Zielkoordinaten, sodass A* das folgende Verhalten aufzeigt):

a) Laufen geht am schnellsten, aber A* untersucht auch einige Pfade, die die Nutzung der U-Bahn beinhalten.
b) Die optimale Lösung beruht auf Fahren ohne Umsteigen.
c) Die optimale Lösung beinhaltet 1x Umsteigen.

Anschließend untersuchen Sie Edge Cases, um die Grenzen der Anwendbarkeit auszuleuchten:

d) Finden Sie ein Koordinatenpaar, für das Ihr Programm aufgrund der Einschränkung (1) mit einer (potenziell) nicht optimalen Lösung terminiert.
e) Finden Sie ein Koordinatenpaar (und ggfs. ein anderses Limit für (1)), für das Ihr Programm aufgrund der Einschränkung (1) ohne Lösung terminiert. Geben Sie zusätzlich zur Lösung den Hinweis "mit der maximalen Pfadlänge kann eine Lösung nicht generiert werden" aus.
f) Finden Sie ein Koordinatenpaar, für das Ihr Programm aufgrund von Einschränkung (2) abbricht.

Falls Sie solche Edge Cases nicht konstruieren können (bitte dokumentieren Sie Ihre Versuche), dann dürfen Sie das Speicherlimit auch auf einen kleineren Wert (z.B. 38) setzen.

Hinweise und Tipps

Sie dürfen KI-Tools (z. B. ChatGPT oder Claude) zur Lösung verwenden. Im Abschnitt „Reflexion" Ihrer schriftlichen Abgabe müssen Sie die KI-Interaktion dokumentieren.

[4] Dann könnte einer der aufgrund der *p_max*-Limitierung nicht expandierten Pfade noch schneller zum Ziel führen.

Ihren Code entwickeln Sie in einem neu anzulegenden Projekt unter git.tu-berlin.de. Unwichtig sind Quellcode-Dokumentation und Unit-Tests, Ihr Programm muss aber korrekt laufen. Wie man Ihr Programm startet, dokumentieren Sie im Readme.md Ihres gitlab-Projekts.

Abgabe der Lösung

Folgende Deliverables müssen Sie erstellen und abgeben (in ISIS hochladen):

1. Report (pdf): Dokumentieren Sie Ihre Lösung. Der Report enthält:
   - Ein Titelblatt mit folgenden Informationen: “Aufgabenblatt 1”, Gruppennummer, Namen der an der Lösung beteiligten Gruppenmitglieder, Links zu den digitalen Abgaben (Code und Screencast), mit Passwortinformationen, sofern zum Zugriff notwendig.
   - Kommentierungen zum Softwaredesign: Wie werden Zustände, Aktionen und Zieltest im Code repräsentiert, und wie haben Sie Ihre Heuristik gestaltet? Wie werden die Einschränkungen und das Logging der am Ende auzugebenden Daten/Zahlen im Code umgesetzt? Wo haben Sie Designentscheidungen getroffen, die die Aufgabenspezifikation oder Lösungen einschränken? Mit welcher Begründung? (Umfang: < 1 Seite)
   - Ihre Use-Cases, die die Korrektheit Ihres Programms dokumentieren: Kurze Darstellung inklusive der optimalen Lösung und ggfs. Notierung eines Pfades, der vom A*-Algorithmus untersucht wird, aber nicht zur Löung beiträgt) (Umfang: jeweils 1 Absatz)
   - Ihre Experimente zu den Edge Cases: Jeweils eine kurze Darstellung und Erklärung, warum Sie diese Probleminstanz gewählt haben. Falls Sie das Speicherlimit herabgesetzt haben, bitte auch dieses dokumentieren und begründen. (Umfang: jeweils 1 Absatz)
   - Reflexion: Dokumentieren Sie den Einsatz von KI während des Prozesses der Aufgabenblattbearbeitung. Welche Aufgaben wurden welcher KI mit welchem Prompt übertragen? Wo waren danach noch Anpassungen nötig? Gegebenenfalls ist auch die Prompt-Historie interessant, diese können Sie gerne digital extra abgeben. (Umfang ≤ 1 Seite)
2. Screen Recording (mp4): Dokumentieren Sie die Lauffähigkeit Ihres Programms, indem Sie durch Screen Recording (ohne Tonspur) die Konsolenausgaben Ihrer Use Cases und Experimente aufzeichnen. Laden Sie den Film in die Tubcloud hoch.

Den Report laden Sie in die ISIS-Aufgabe "Aufgabenblatt 1" hoch. Links zu Ihrem git-Repository und dem Screen Recording geben Sie als Kommentar ebenfalls in der ISIS-Aufgabe ab. Falls der Zugang passwortgeschützt ist, geben Sie auch das Passwort im Kommentar mit an. Wenn Sie mehrere Lösungen abgeben, wird der letzte Upload vor dem Abgabetermin bewertet.

Bewertung des Aufgabenblattes

Sie erhalten eine Bewertung auf der 100er-Punkteskala. Die Verrechnung zu Portfoliopunkten erfolgt am Ende des Semesters durch Aufsummierung Ihrer Aufgabenblätterpunkte und anschließende Division.

Die volle Punktzahl erhalten Sie, wenn die geforderten Deliverables zum Ablauf der Deadline in zufriedenstellender Qualität vorliegen. Abzüge von jeweils 20% gibt es für verspätete Einsendungen, oder wenn Ihre Lösungen teilweise nicht zugreifbar sind. Wenn Sie kein lauffähiges Programm abgeben, dürfen Sie im Nachgang (2 Tage Zeit hierfür) die Experimente durch eine KI analysieren lassen oder als Gedankenexperiment analysieren (eine vollständige Simulation ist nicht erforderlich, aber eine Analyse des vermuteten Verhaltens des Algorithmus). Damit können Sie noch auf maximal 70% der Punkte kommen. Auslassungen, Fehler oder fehlende Erklärungen im Report führen ebenfalls zu Abzügen.

Erklärung der Tabellenspalten und -zeilen der csv-Datei

Das Berliner U-Bahnnetz wird in einer Adjazenzmatrix repräsentiert. Für jeden U-Bahnhof sind pro Linie die (maximal) 2 direkt erreichbaren Nachbarbahnhöfe notiert. Für Umsteigebahnhöfe zusätzlich die sich kreuzenden Linien.

- Zeile 1: Spaltenüberschriften
- Spalten from_lat, from_lon , to_lat, to_lon enthalten die Geokoordinaten (Latitude/ Longitude) von Start- und Zielort.
- Zeilen 2 – 365: alle direkt benachbarten U-Bahnstationen aller Linien (Spalten *from, to, line*). Spalte *action* hat den Wert *travel*, Spalte *cost_min* enthält die Fahrdauer in Minuten. Spalte *station* ist leer, hier werden nur Umsteigebahnhöfe für Umsteigeaktionen modelliert (siehe unten)
- Zeilen 366 – 431: Umsteigemöglichkeiten auf den Bahnhöfen, wo sich Linien kreuzen. Spalte *action* hat den Wert *transfer*, Spalte *cost_min* enthält die pauschale Umsteigezeit von 5 Minuten, Spalte *station* den Namen des U-Bahnhofs. Die Geokoordinaten der Umsteigeaktionen sind natürlich identisch für Start- und Zielort, weil nur innerhalb des Bahnhofs gelaufen wird, was pauschal mit der Umsteigezeit verrechnet wird.